StreamCab: Real-Time Taxi Fare Intelligence with Kafka, Spark, and XGBoost

Your Name
Group Member 1
Group Member 2
Group Member 3

April 17, 2026

Abstract

This paper presents StreamCab, an end-to-end data engineering and machine learning system for real-time taxi analytics and short-term fare prediction. The pipeline replays historical New York City taxi trips into Apache Kafka, processes and aggregates the stream using Apache Spark Structured Streaming, stores results in PostgreSQL, and serves predictions through a dashboard interface. We describe the architecture, data processing methodology, model training strategy, and performance outcomes, then discuss practical lessons learned about reliability, memory constraints, and deployment trade-offs in single-node environments.

1 Introduction

Urban transportation data is high-volume, time-sensitive, and geographically heterogeneous. Static batch reports are often insufficient for operational decisions such as where demand is increasing and what pricing behavior to expect in the next few minutes. This project investigates whether a compact local stack can deliver near real-time analytics and prediction from historical taxi records replayed as live events.

The objectives of StreamCab are:

- Build a reproducible streaming pipeline from data ingestion to visualization.
- Produce interpretable zone-level aggregate metrics over short windows.
- Train and serve a practical fare predictor under memory constraints.
- Evaluate model quality against a baseline and report engineering trade-offs.

2 Related Work / Background

Streaming architectures commonly use Kafka for event transport, Spark Structured Streaming for stateful transformations, and SQL databases for serving curated outputs [2, 3]. Gradient-boosted trees, especially XGBoost, are widely used for tabular prediction due to strong accuracy-speed trade-offs [4].

Compared to distributed cloud deployments, this project emphasizes local reproducibility and careful memory-aware design. The training pipeline uses batched parquet processing and continued model updates to support large datasets on limited RAM.

3 System Architecture and Design

3.1 Architecture Overview

The system has six services: PostgreSQL, Kafka (+ producer), Spark/predictor/dashboard app container, consumer, one-shot trainer, and Kafka Control Center.

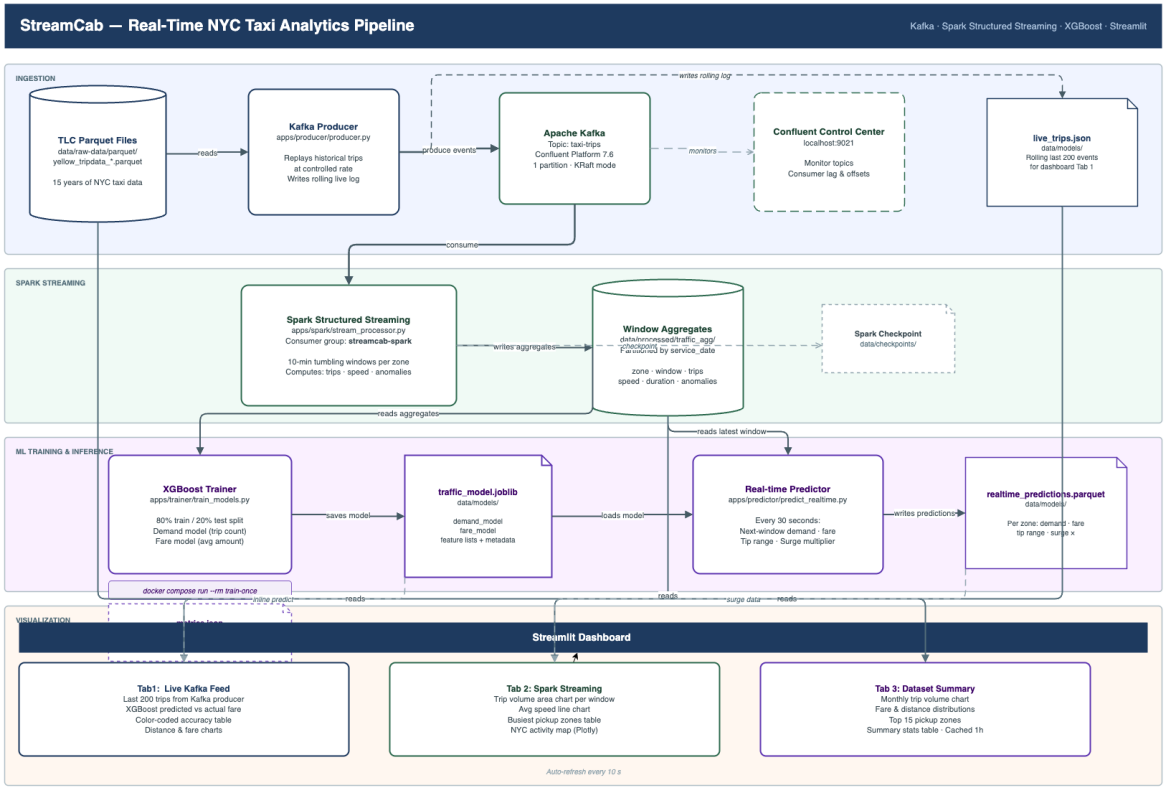

Figure 1: StreamCab end-to-end architecture.

3.2 Component Responsibilities

- **Producer:** Streams normalized trip events from parquet in replay loops.
- **Kafka:** Buffers and distributes trip events to downstream consumers.
- **Spark Structured Streaming:** Applies cleaning rules and windowed zone aggregations.
- **PostgreSQL:** Stores live trips, traffic aggregates, and prediction outputs.
- **Trainer:** Builds XGBoost fare model from historical aggregates.
- **Predictor + Dashboard:** Performs periodic inference and displays live analytics.

3.3 Tools Not Covered in Class (Brief Intro)

If needed for course context:

- **Apache Kafka:** Distributed event log for pub/sub data streaming.
- **Spark Structured Streaming:** Incremental DataFrame processing with watermarks and windows.
- **JDBC/Psycopg integration:** Database connectivity between Spark/Python services and PostgreSQL.

4 Data Collection and Processing

4.1 Data Source

Primary data comes from publicly available NYC TLC yellow taxi trip records (monthly parquet files).

Data source link: `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page` [1]

4.2 Collection Procedure

1. Download selected monthly parquet files.
2. Store files under project raw-data directory.
3. Replay rows as pseudo-live events through Kafka.

4.3 Schema and Metadata

Raw/event schema (core fields):

- pickup_datetime, dropoff_datetime

- passenger_count, trip_distance
- pu_location_id, do_location_id
- total_amount, emitted_at

Windowed aggregate schema:

- window_start/window_end, pu_location_id, trip_count
- avg_speed_mph, avg_duration_min, avg_trip_distance
- avg_total_amount, anomaly_count

4.4 Processing Method (Inputs and Outputs)

Inputs are raw parquet trip files and zone centroid metadata. Processing includes type normalization, quality filters, temporal feature extraction, and sliding-window aggregation. Outputs include:

- Streaming aggregate table for current operational state.
- Trained fare model artifact and evaluation metrics.
- Prediction records consumed by dashboard visualizations.

5 Model Training and Inference Strategy

5.1 Baseline and Features

The baseline predicts mean fare by pickup zone and hour. The XGBoost model uses zone ID, hour/day/weekend flags, average distance, duration, speed, and trip count.

5.2 Memory-Aware Training

To support large datasets on limited memory, the trainer:

- reads parquet in bounded batches,
- computes additive aggregate statistics,
- finalizes means after grouped reductions,
- applies continued training across chronological batches,
- selects the best validation checkpoint for final artifact.

6 Results and Analysis

6.1 Quantitative Results

Replace or update with your final run metrics.

Model	MAE	MAPE (%)
Baseline (zone-hour mean)	1.9891	15.43
XGBoost (test)	0.4851	3.42

Table 1: Example model performance summary.

6.2 Dashboard and Analytics Outputs

Include screenshots or figures for:

- Live trips per minute
- Zone ranking and surge signals
- City map activity visualization

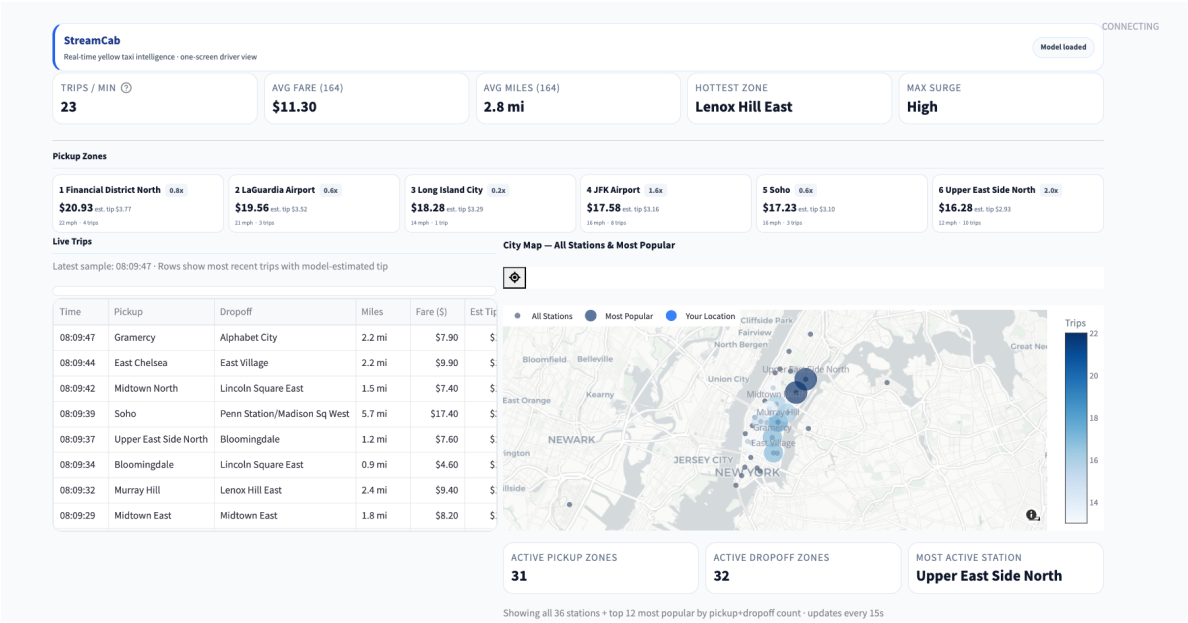

Figure 2: StreamCab dashboard showing live trip table, zone-level insights, and map-based analytics.

6.3 Discussion

The pipeline demonstrates that practical near real-time analytics can be achieved in a local setup when ingestion, processing, and training are designed for bounded memory and idempotent writes.

SQL-side narrowing for predictor reads and stream checkpointing significantly improve operational stability.

6.4 Limitations and Threats to Validity

Although the quantitative results are strong, several limitations affect generalization and operational confidence. First, the stream is a replay of historical data rather than a true live feed, so observed latency and throughput may differ from production systems with bursty arrival patterns, schema drift, and upstream outages. Second, zone-level aggregates improve model stability but can hide fine-grained effects such as street-level congestion, event-driven anomalies, and rare pickup/dropoff combinations.

The evaluation window is also constrained by selected months and filtering rules. If market conditions shift (fuel prices, policy changes, seasonality, or weather shocks), model error can increase without immediate visibility. Future iterations should include explicit drift indicators (feature distribution shift, residual monitoring, and alarm thresholds) to ensure model performance remains within acceptable bounds. Finally, local single-node deployment simplifies reproducibility, but it does not fully represent fault domains and scaling behavior in distributed environments.

6.5 Reproducibility and Operational Notes

To support reproducibility, all services are containerized and orchestrated through Docker Compose with deterministic startup dependencies. The project separates concerns across producer, streaming processor, consumer, trainer, and dashboard services. This isolation reduces coupling and makes it easier to troubleshoot failures by layer. The end-to-end runbook consists of: (1) downloading raw TLC parquet data, (2) launching services, (3) running one-shot training, (4) starting replay/streaming, and (5) validating outputs in PostgreSQL and the dashboard.

Operationally, two implementation choices were especially important. The first is checkpointed stream processing with bounded windows, which limits duplicate writes and supports restart safety. The second is memory-aware training via batched parquet reads and incremental model updates, which enables full-dataset learning on commodity hardware. For teams extending this project, we recommend adding lightweight data quality contracts at ingest time (null/negative checks, timestamp sanity, and schema assertions) and documenting expected service-level objectives for latency and prediction freshness.

The architecture can be extended incrementally: move stateful processing to a managed cluster, introduce model registry/versioning, and add automated backtesting before promoting new models. These steps preserve the current educational value while improving production readiness.

7 Conclusion and Key Takeaways

What worked well:

- Modular services and reproducible local orchestration.
- Streaming aggregates as robust prediction features.
- Strong tabular prediction performance with XGBoost.

Main challenges:

- Large data volume vs local memory constraints.
- Reliability semantics across stream and database boundaries.
- Balancing latency, correctness, and simplicity.

Future work:

- Add drift monitoring and automated retraining schedules.
- Extend features with weather/events/geospatial context.
- Compare distributed multi-node deployment variants.

Submission Checklist Mapping

- GitHub repo with README walkthrough: **Yes** (update final links).
- Public data source link: **Include in README and this paper**.
- Research-style report (6–8 pages): **This report compiles within target range**.
- Group and author names: **Replace placeholders on title page**.

References

[1] NYC Taxi & Limousine Commission, *TLC Trip Record Data*, `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`

[2] Apache Kafka Documentation, `https://kafka.apache.org/documentation/`

[3] Apache Spark Structured Streaming Guide, `https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html`

[4] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, KDD 2016.